한국성서대학교
KOREAN BIBLE UNIVERSITY

「2026년도 2학기 한국성서대학교 AI융합학부」

AIoT프로그래밍

아두이노 통신모듈
: RFID RC522 RF, 적외선 통신모듈 실습

11주차





01 RFID 통신

RFID RC522 RF **통신모듈 실습**

- RFID 통신 모듈 제어에 대하여 알아본다.
- RFID 카드가 RFID-RC522RF 모듈이 감지될 때마다 RFID 카드의 고유 식별 번호를 인식해 시리얼 모니터로 출력하는 동작을 수행
- RFID (Radio Frequency Identification):
  무선 주파수를 이용해 태그(Tag) 정보를 인식하는 기술.
  → 카드나 태그에 저장된 고유 식별번호를 비접촉 방식으로 읽음.

RFID RC522 RF **통신모듈 실습**

- RFID-RC522 모듈 구성품
  - RFID-RC522 RF 모듈
  - RFID 카드 / 열쇠고리형 태그
- 작동 원리

  리더기(RC522)가 RF 신호를 송신 → 태그의 IC칩이 응답 → 리더기가 정보를 수신.

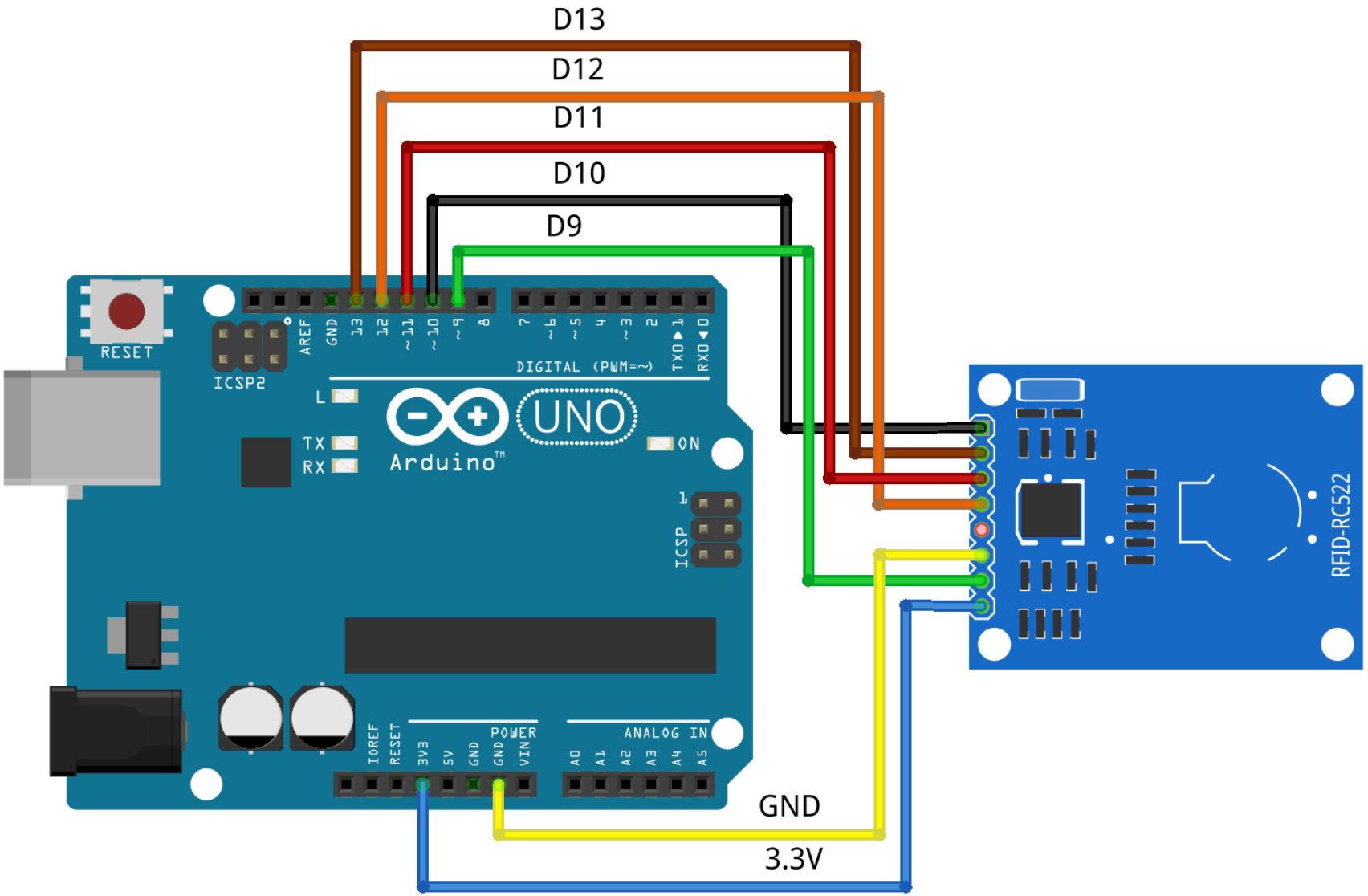
RFID RC522 RF **통신모듈 실습**

핀 이름	의미	설명
SDA	Serial Data	SPI 모드에서 슬레이브 선택 신호로 사용됨.
SCK	Serial Clock	데이터 전송 동기화를 위한 클록 신호.
MOSI	Master Out Slave In	마스터(아두이노)가 슬레이브(RC522)에게 데이터 전송하는 출력핀
MISO	Master In Slave Out	슬레이브에서 마스터로 데이터 전송.
IRQ	Interrupt Request	인터럽트 발생 시 마이크로컨트롤러에 알림.
RST	Reset	모듈 재설정용 핀.
3.3V	전원	3.3V 전원 공급.
GND	접지	회로의 기준 전압 연결.

RFID RC522 RF 통신
– SPI(Serial Peripheral Interface) 인터페이스 특징

- **전이중 통신**
  - SPI는 전이중 통신(데이터를 동시에 송신과 수신할 수 있는 통신 방식)을 지원하여 데이터를 송신하면서 동시에 데이터를 수신
- **마스터 슬레이브 구조**
  - SPI는 하나의 마스터와 하나 이상의 슬레이브로 구성. 마스터가 통신을 제어하고 슬레이브는 마스터의 클록에 동기화되어 동작
- **시리얼 데이터 전송**
  - SPI는 데이터가 한 비트씩 직렬로 전송되는 시리얼 통신 방식
- **클럭 신호(동기식)**
  - SPI 통신은 클록 신호를 사용하여 데이터의 송수신 타이밍을 맞추는 동기식 통신(8 Mbit/s)
- **다중 슬레이브 지원**
  - SPI는 하나의 마스터가 여러 개의 슬레이브를 제어, 각 슬레이브는 별도의 선택 신호를 통해 개별적으로 통신
- **CS(Chip Select) 신호**
  - 마스터는 통신할 슬레이브를 선택하기 위해 CS(Chip Select) 신호를 사용. 선택된 슬레이브만이 마스터와 통신을 수행. 여러 개의 슬레이브 중 “지금 너랑 통신할 거야!” 하고 마스터가 특정 슬레이브를 선택하는 신호

RFID RC522 RF통신모듈 실습 회로도

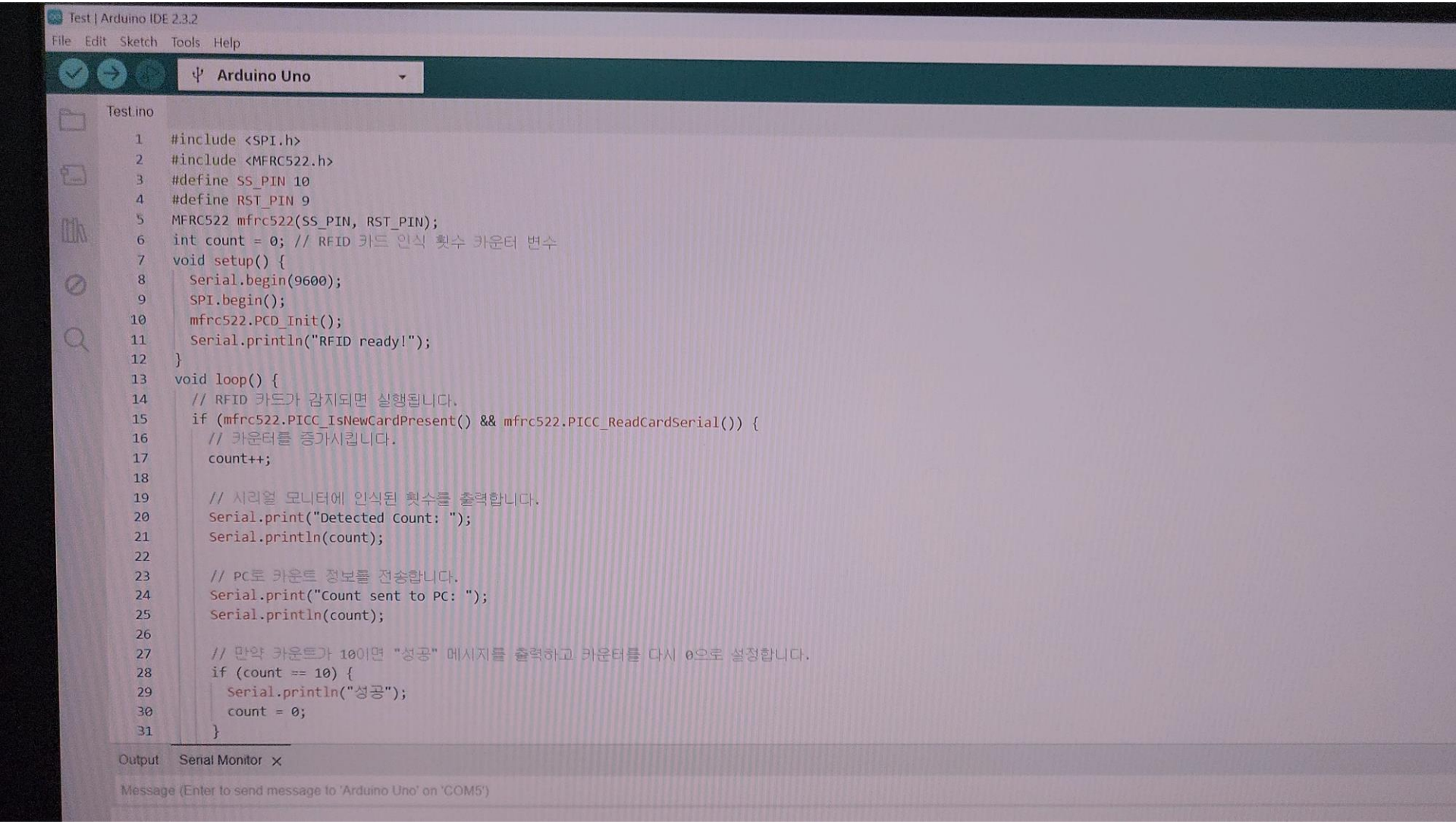


RFID RC522 RF**통신모듈 실습 회로도**

RC522 핀	아두이노 UNO 핀
SDA : SS(Slave Select) 핀	10
SCK	13
MOSI	11
MISO	12
RST	9
3.3V	3.3V (RF 회로는 3.3V 동작에 최적화됨)
GND	GND

```cpp
#include <SPI.h> // SPI (Serial Peripheral Interface) 통신을 사용하기 위해 SPI 라이브러리를 불러옴
#include <MFRC522.h>  // RFID 리더 MFRC522 전용 라이브러리를 불러옴
#define SS_PIN 10 // SPI 통신에서 슬레이브 선택선(SS, Slave Select) 에 해당하는 핀 번호를 정의
#define RST_PIN 9 // MFRC522 모듈의 리셋(Reset) 핀을 Arduino의 9번 핀에 연결했다고 정의
MFRC522 mfrc522(SS_PIN, RST_PIN); // MFRC522 클래스로 객체 mfrc522를 생성
int count = 0; // RFID 카드 인식 횟수 카운터 변수
void setup() {
  Serial.begin(9600);
  SPI.begin();// MOSI, MISO, SCK 핀을 따로 설정하지 않음. SPI.begin();함수가 자동으로 설정
  mfrc522.PCD_Init();
  Serial.println("RFID ready!");
}
void loop() {
  // RFID 카드가 감지되면 실행
  if (mfrc522.PICC_IsNewCardPresent() && mfrc522.PICC_ReadCardSerial()) {
    //PICC_IsNewCardPresent()는 안테나 근처에 카드가 있는지 확인, 1: 감지, 0 : 미감지
    //PICC_ReadCardSerial() 현재 감지된 카드로부터 UID(시리얼/고유번호)를 읽어 mfrc522.uid 구조체(예: mfrc522.uid.uidByte[], mfrc522.uid.size)에 저장, 1: 감지, 0 : 미감지
    count++;     // 카운터를 증가
    Serial.print("Detected Count: ");     // 시리얼 모니터에 인식된 횟수를 출력
    Serial.println(count);
    // PC로 카운트 정보를 전송
    Serial.print("Count sent to PC: ");
    Serial.println(count);
    // 만약 카운트가 10이면 "성공" 메시지를 출력하고 카운터를 다시 0으로 설정
    if (count == 10) {
      Serial.println("성공");
      count = 0;
    }
    delay(1000);     // 다음 카드 감지를 위해 잠시 대기
  }
}
```

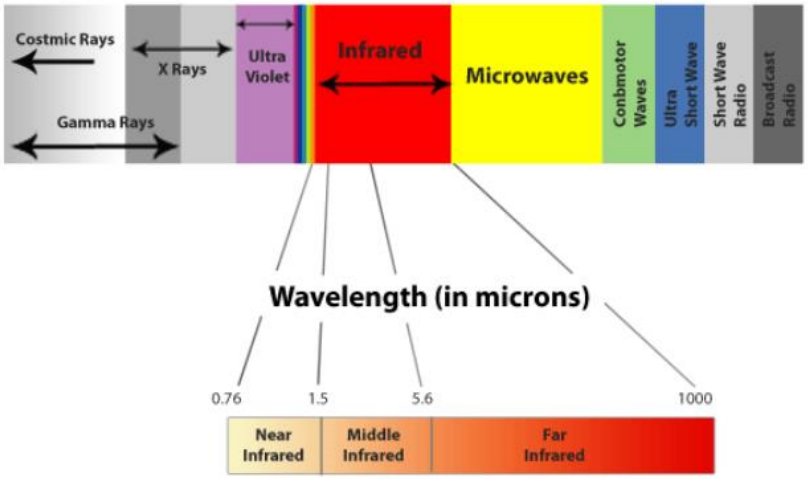
RFID RC522 RF통신모듈 실습 영상



02 적외선 통신

적외선 통신

- 적외선 IR(Infrared)-가시광선보다 파장이 긴 것이 특징



<적외선의 파장은 0.75um부터 1,000um사이에 위치합니다>





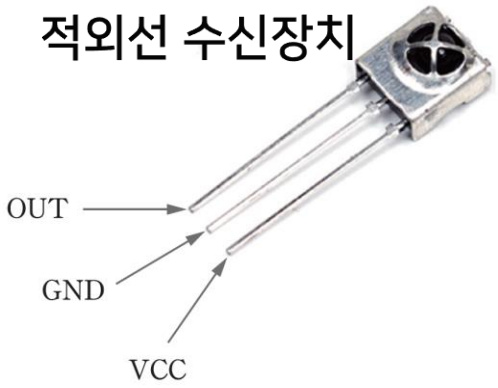
아두이노 적외선 통신 사용

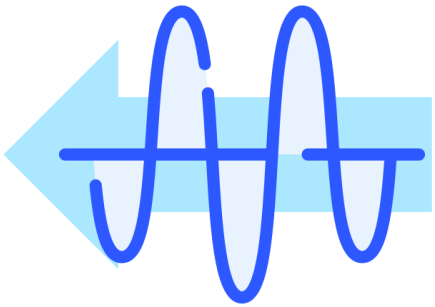
•리모컨은 **IR LED를 통해 신호를 전송**하며, 단순한 On/Off 방식만으로는 **주변 조명이나 햇빛 등 적외선 잡음에 의해 수신 오류**가 발생할 수 있음.
•이를 방지하기 위해 캐리어 주파수(carrier frequency, 보통 38kHz)를 사용하며, 적외선을 **고속으로 깜빡이면서** 그 깜빡임 신호에 데이터를 실어 전송함.

- 근거리 무선 통신을 위해 흔히 사용
  - 송수신 거리가 짧고 송신기와 수신기가 마주보고 있어야함(단방향)
  - 대부분의 리모컨에서 사용
- 38KHz 캐리어 주파수를 사용
  - 자연계에서는 38KHz 주파수가 흔하지 않음
- 간단하고 저렴하게 무선 통신 구현 가능



적외선 수신장치







리모콘 코드표



FFA25D FF629D FFE21D

FF22DD FF02FD FFC23D

FFE01F FFA857 FF906F

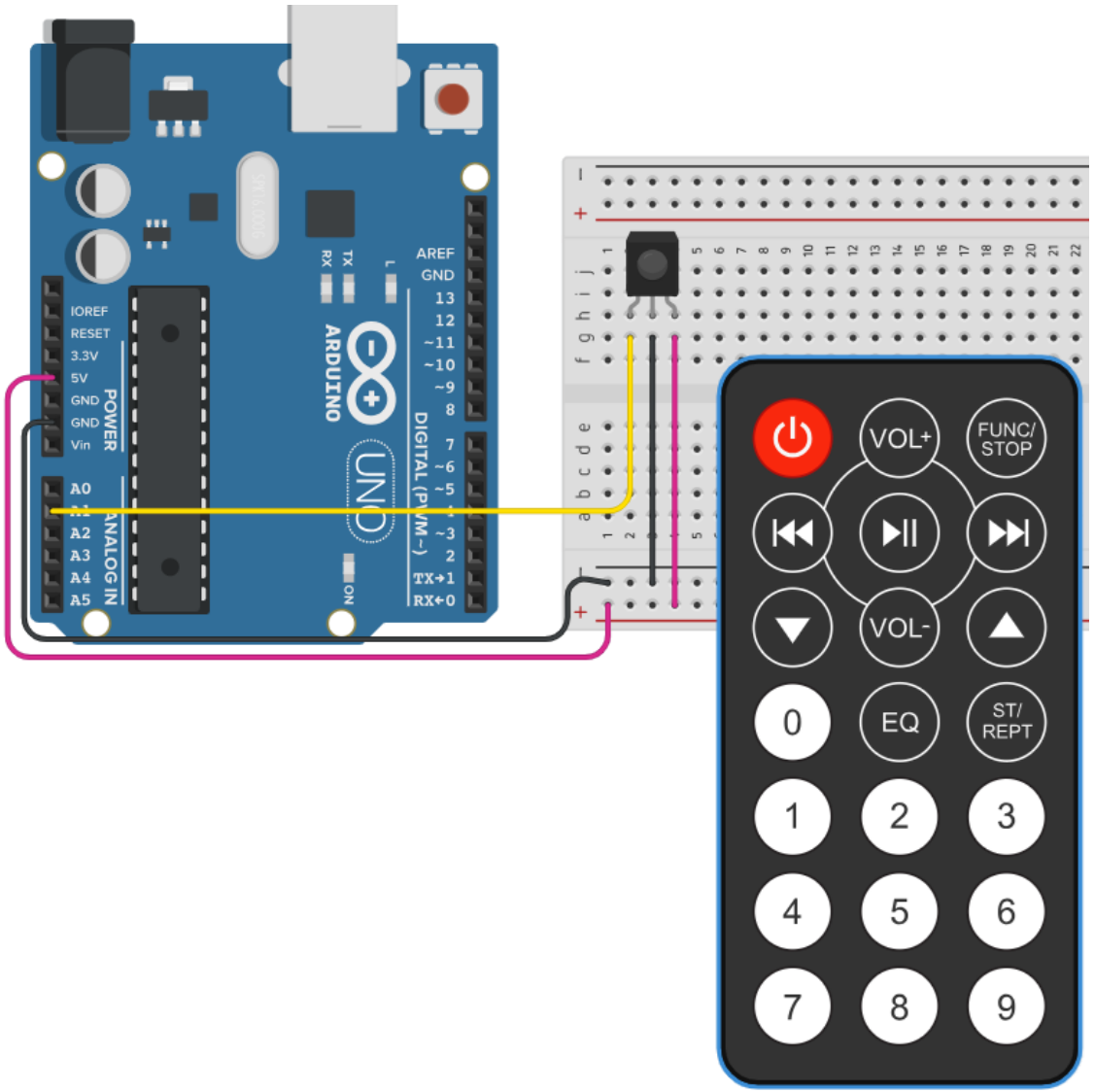
FF6897 FF9867 FFB0F4

FF30CF FF18E7 FF7A85

FF10EF FF38C7 FF5AA5

FF42BD FF4AB5 FF52AD

적외선 통신 회로도


IOREF
RESET
3.3V
5V
GND
GND
Vin
POWER
A0
A2
A3
A4
A5
ANALOG IN
ARDUINO
UNO
AREF
GND
13
12
~11
~10
~9
8
7
~6
~5
4
~3
2
TX→1
RX←0
DIGITAL (PWM~)
VOL+
FUNC/
STOP
VOL-
EQ
ST/
REPT
0
1
2
3
4
5
6
7
8
9

uno13_1.ino : 리모컨 버튼 16진수 값 확인(제조사 마다 다름)

[New 4.x program]

```
#include <IRremote.hpp> // IRremote 라이브러리 포함
#define IR_PIN A1 // 적외선 수신기를 연결할 핀을 A1으로 설정

void setup() {
  Serial.begin(9600); // 시리얼 통신초기화

  // IR_PIN에 연결된 핀에서 신호를 수신
  // ENABLE_LED_FEEDBACK:수신되면 default LED가 깜빡임
  IrReceiver.begin(IR_PIN, ENABLE_LED_FEEDBACK);// 적외선 수신기 초기화
}

void loop() {
  if (IrReceiver.decode()) {  // 적외선 데이터가 수신되었는지 확인

      // 수신된 데이터를 16진수 형식으로 시리얼 모니터에 출력
      Serial.println(IrReceiver.decodedIRData.decodedRawData, HEX);

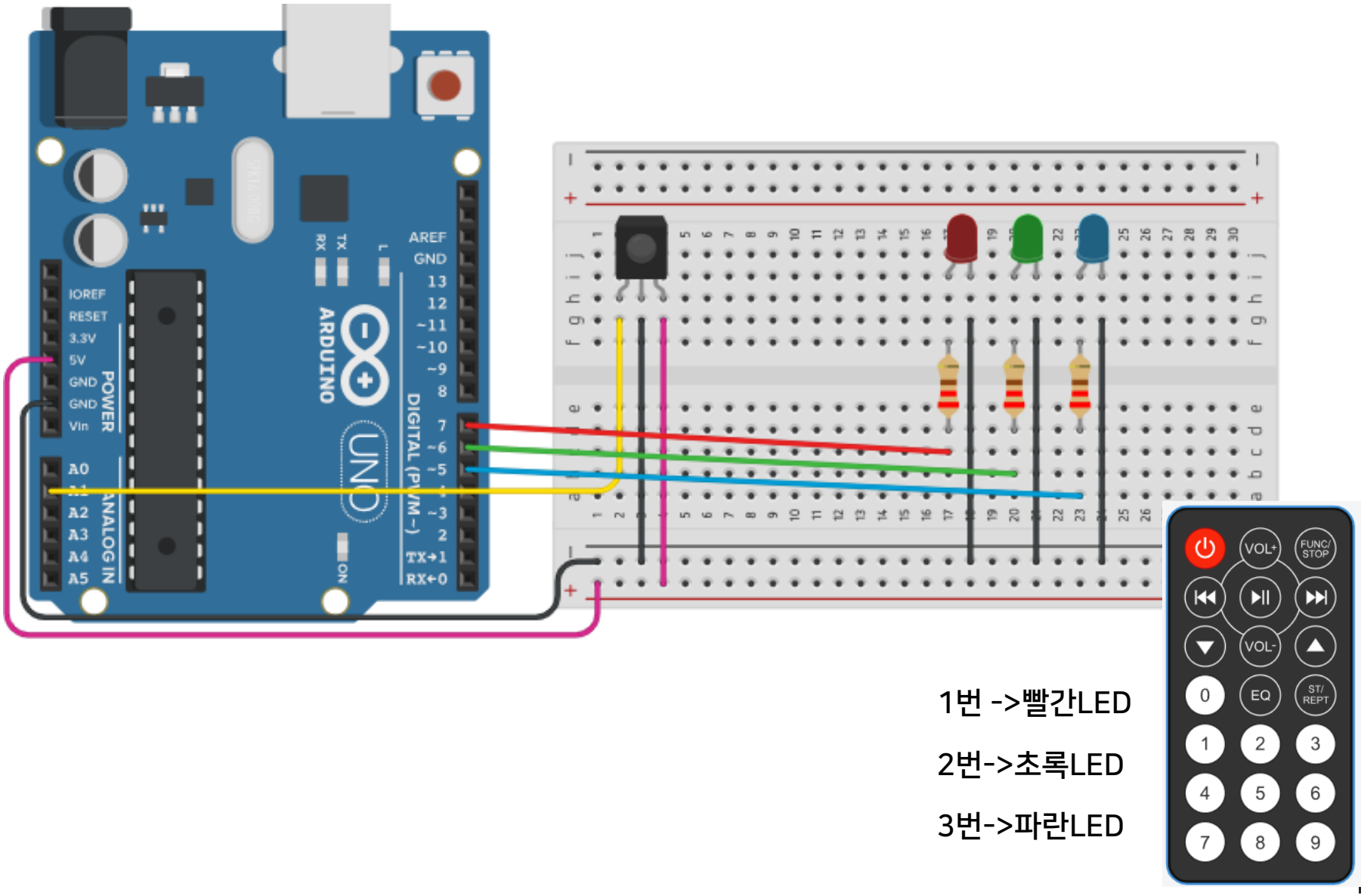
      IrReceiver.resume(); // 다음 신호를 받을 준비
  }
}
```

출력 시리얼 모니터 ×

Message (Enter to send messa

```
21:05:30.106 -> EA15FF00
21:05:30.959 -> EA15FF00
21:05:31.812 -> E619FF00
21:05:32.830 -> F20DFF00
21:05:33.723 -> A15EFF00
21:05:35.330 -> E718FF00
21:05:37.033 -> E916FF00
```

적외선 통신 회로도 LED 제어



1번 ->빨간LED

2번->초록LED

3번->파란LED

Uno13_2.ino :uno13_1.ino에서 추가

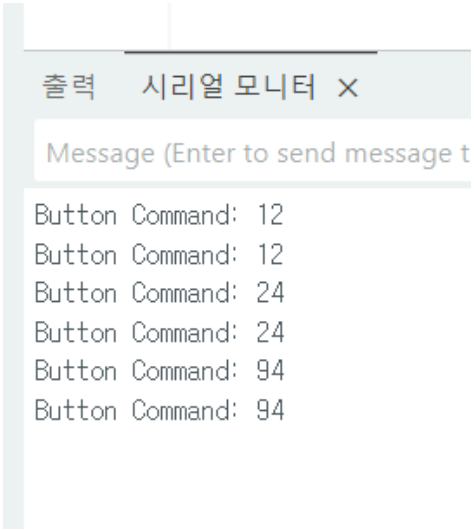
- 전역변수

```
int LED[]={7,6,5};
```

- ledAllOff() 함수 추가

```
void ledAllOff(){
  for(int i=0; i<3; i++)digitalWrite(LED[i], 0);
}
```

- 리모컨의 command값 확인




```
#include <IRremote.hpp>

void setup() {
  Serial.begin(9600);
  IrReceiver.begin(A1);
}

void loop() {
  if (IrReceiver.decode()) {
    Serial.print("Button Command: ");
    Serial.println(IrReceiver.decodedIRData.command);
    IrReceiver.resume();
  }
}
```

Uno13_2.ino :uno13_1.ino에서 추가

- LED 제어 프로그램

switch 각 요소는 버튼 커멘드 값으로 설정

```
#include <IRremote.h> // 적외선 수신 라이브러리 포함
#include <IRremote.hpp>

int LED[] = {7, 6, 5};  // LED 핀 배열
int button;             // IR 버튼 값 저장 변수

// 모든 LED 끄기 함수
void ledAllOff() {
  for (int i = 0; i < 3; i++) {
    digitalWrite(LED[i], LOW);
  }}

void setup() {
  Serial.begin(9600);
  IrReceiver.begin(A1); // IR 수신기 연결 핀
  for (int i = 0; i < 3; i++) {
    pinMode(LED[i], OUTPUT);
  }
}
```

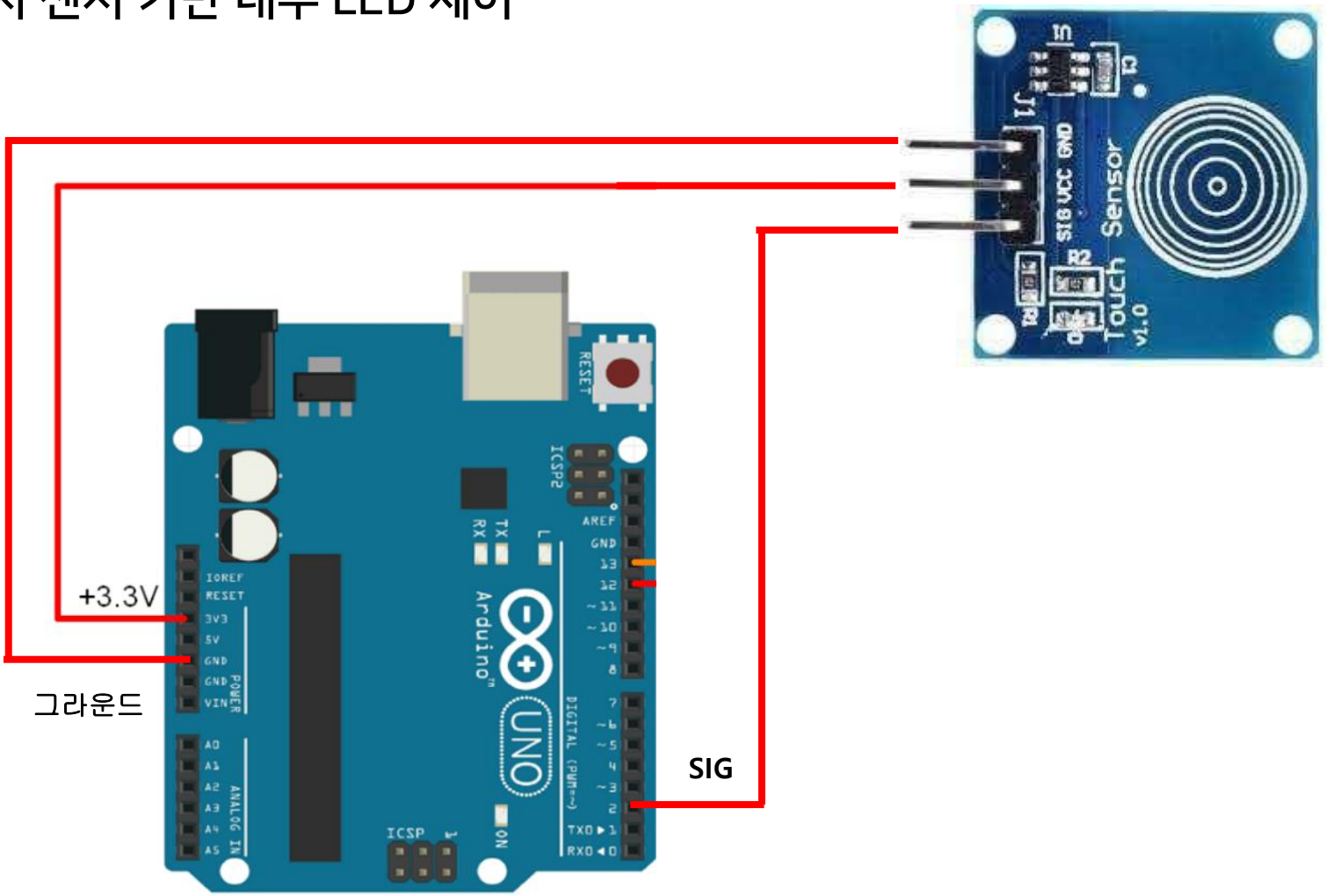
```
void loop() {
  if (IrReceiver.decode()) {
    button = IrReceiver.decodedIRData.command;
    ledAllOff();

    switch (button) {
      case 12:
        digitalWrite(LED[0], HIGH);
        break;
      case 24:
        digitalWrite(LED[1], HIGH);
        break;
      case 94:
        digitalWrite(LED[2], HIGH);
        break;
    }
    IrReceiver.resume(); // 꼭 추가
    delay(100);          // LED 유지 시간
  }
}
```

03 터치 센서

터치 센서 실습

- 터치 센서 기반 내부 LED 제어



터치 센서 실습

LED 제어 프로그램

1. 터치 센서와 **LED** 핀을 초기화함.
2. 터치 센서의 출력값을 읽음 **(digitalRead).**
3. 터치 상태가 **HIGH**인지 확인함.
4. **HIGH**이면 **LED**를 켜고 **Touch Detected!** 메시지를 출력함.
5. **LOW**이면 **LED**를 끔.
6. 다시 **2**번으로 돌아가 반복 실행함.

```
1  #define TOUCH_PIN 2
2  #define LED_PIN 13
3
4  void setup() {
5
6    Serial.begin(9600);
7
8    pinMode(TOUCH_PIN, INPUT);
9    pinMode(LED_PIN, OUTPUT);
10
11 }
12
13 void loop() {
14
15   int touchState = digitalRead(TOUCH_PIN);
16
17   if (touchState == HIGH) {
18
19     digitalWrite(LED_PIN, HIGH);
20     Serial.println("Touch Detected!");
21
22   }
23   else {
24
25     digitalWrite(LED_PIN, LOW);
26
27   }
28
29 }
```

터치 센서 실습

LED 제어 프로그램

1. 터치 센서와 **LED** 핀을 초기화함.
2. 이전 터치 상태를 **LOW**로 초기화함 (**previousState = LOW**).
3. 현재 터치 센서의 출력값을 읽음.
4. 이전 상태와 현재 상태를 비교함.
5. 이전 상태가 **LOW**이고 현재 상태가 **HIGH**이면 **LED**를 켜고 메시지를 한 번 출력함.
6. 현재 상태가 **LOW**이면 **LED**를 끔.
7. 현재 상태를 이전 상태 변수에 저장함.
8. 다시 3번으로 돌아가 반복 실행함.

```
1  #define TOUCH_PIN 2
2  #define LED_PIN 13
3
4  int previousState = LOW;
5
6  void setup() {
7
8    Serial.begin(9600);
9
10   pinMode(TOUCH_PIN, INPUT);
11   pinMode(LED_PIN, OUTPUT);
12
13 }
14
15 void loop() {
16
17   int touchState = digitalRead(TOUCH_PIN);
18
19   // 이전 상태가 LOW이고 현재 상태가 HIGH일 때
20   if (previousState == LOW && touchState == HIGH) {
21
22     digitalWrite(LED_PIN, HIGH);
23
24     Serial.println("Touch Detected!");
25
26   }
27
28   // 손가락을 떼면
29   if (touchState == LOW) {
30
31     digitalWrite(LED_PIN, LOW);
32
33   }
34
35   // 현재 상태를 이전 상태로 저장
36   previousState = touchState;
37
38 }
```

Thank You